PvrGPU Gallium Driver — Block Diagram, Data Path & Pseudocode

本文件以 block diagram、data path flow 及 pseudocode 形式詳述 `src/gallium/drivers/pvrgpu/` Gallium driver 的行為。

此 driver 是一個 Mesa Gallium 實作，將 OpenGL ES API call 翻譯成 `pvrgpu.driver-command.v1` 格式的指令，交給 SystemC model 執行。

---

0. Top-Level Architecture Block Diagram

PvrGPU Gallium Driver — Architecture Overview
Mesa Gallium pipe_screen / pipe_context 實作
將 GLES API call 轉成 driver command 給 SystemC model

GLES Application
glClear, glDrawArrays, glDrawElements, glTexImage2D, etc.

↓ Mesa state tracker (GLES -> Gallium)

Mesa Gallium State Tracker
pipe_context->clear(), draw_vbo(), set_*(), flush()

↓ Gallium pipe interface

PvrGPU Gallium Driver

```
+-----------------------------------------------------+
| pvrgpu_screen.c — pipe_screen 實作                  |
| pvrgpu_context.c — pipe_context 實作 (draw/clear)   |
| pvrgpu_state.c — 固定功能 state object 管理         |
| pvrgpu_resource.c — buffer/texture 記憶體管理       |
| pvrgpu_clear.c — color/depth/stencil clear          |
| pvrgpu_pco.c — PCO shader compiler 橋接             |
| pvrgpu_cmd.c — driver command 序列化 + SystemC      |
| pvrgpu_counter.c — driver event counter 記錄        |
+-----------------------------------------------------+
```

↓ driver command (text file 或 SystemC C API)

Output Channels (擇一或同時)
1. PVRGPU_DRIVER_COMMAND_OUT -> driver-command.txt
2. PVRGPU_SYSTEMC_API_LIB -> dlopen -> SystemC model
3. PVRGPU_DRIVER_COUNTER_OUT -> driver-counter.txt

SystemC Model (pvrgpu-model-stub)
Geometry Pipeline -> Fragment Pipeline -> DRAM readback

---

1. Module Responsibility Map

檔案	行數	角色	職責
`pvrgpu_screen.c`	437	`pipe_screen`	裝置能力查詢、format 支援、shader caps、fence no-op、context factory

pvrgpu_context.c	8120	pipe_context	draw_vbo 分派、draw profile 匹配、clear emit、flush、framebuffer state
pvrgpu_state.c	1305	State objects	blend/DSA/rasterizer/sampler/shader/vertex-elements create/bind/delete + counter
pvrgpu_resource.c	2445	Resources	buffer/texture create、CPU-backed storage、map/unmap、transfer、blit、copy
pvrgpu_clear.c	860	Clear	full-frame color clear -> driver command emit、depth/stencil clear tracking
pvrgpu_pco.c	2076	PCO compiler	NIR -> PCO (gx6250) 編譯、profile 匹配 (conditionals/lit_mesh/texture)
pvrgpu_cmd.c	1436	Command I/O	driver command 序列化成 text 或 SystemC C API submit
pvrgpu_counter.c	55	Counter	JSONL event append 到 PVRGPU_DRIVER_COUNTER_OUT
pvrgpu_systemc_api.h	163	SystemC ABI	C ABI struct 定義，dlopen 動態載入 SystemC model

2. Data Path Flow — 從 GLES 到 SystemC

GLES Application
glClearColor(r,g,b,a); glClear(GL_COLOR_BUFFER_BIT);
glDrawArrays(GL_TRIANGLES, 0, 3);
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_SHORT, indices);
glTexImage2D(...); glBindTexture(...);
glUniform4fv(...);

↓ Mesa state tracker translates to Gallium API

pipe_context->clear(buffers, color, depth, stencil)
pipe_context->draw_vbo(info, indirect, draws, num_draws)
pipe_context->set_framebuffer_state(fb)
pipe_context->set_constant_buffer(stage, index, cb)
pipe_context->create_sampler_state(state)
pipe_context->bind_sampler_states(stage, start, count, samplers)
pipe_context->set_sampler_views(stage, start, count, views)
pipe_context->create_vs_state(shader) / create_fs_state(shader)
pipe_context->create_vertex_elements_state(num, elements)
pipe_context->set_vertex_buffers(start, count, buffers)
pipe_context->flush(fence, flags)

pvrgpu_state.c — State Object Management
W: pvrgpu_blend_state, pvrgpu_depth_stencil_alpha_state,
pvrgpu_rasterizer_state, pvrgpu_sampler_state,
pvrgpu_shader_state (NIR/TGSI), pvrgpu_vertex_elements_state
R: 所有 state 在 draw/clear 時被消費
Counter: create_*/bind_*/delete_* events

pvrgpu_resource.c — Resource Management
create_resource: CPU malloc for buffer/texture
buffer_map / texture_map: 直接回傳 data pointer
buffer_subdata / texture_subdata: memcpy upload
resource_copy_region / blit: CPU-side copy + format convert
Counter: resource_create/map/unmap/transfer/copy events

pvrgpu_clear.c — Clear Path
R: framebuffer state, clear color/depth/stencil values
Decision: full-frame color clear? -> emit driver command
partial/scissored clear -> counter only (unsupported)
W: pvrgpu_clear_color_command -> pvrgpu_cmd.c

pvrgpu_context.c — Draw Path (draw_vbo dispatch)
R: all bound state objects, framebuffer, vertex buffers, indices
Decision tree (fail-closed pattern matching):
1. Suppressed draw? -> counter only
2. Lit-mesh PCO profile match? -> emit PCO triangles command
3. Texture PCO profile match? -> emit PCO triangles command
4. Conditionals PCO profile match? -> emit PCO triangles command
5. Textured triangles (glmark2 effect2d)? -> emit textured cmd
6. Primitive sequence (dEQP)? -> emit counter sequence cmd
7. Array triangle (3v non-indexed)? -> emit triangle command
8. Indexed quad (4v indexed)? -> emit indexed quad command
9. Indexed triangle? -> counter only (not yet lowered)
10. All else -> unsupported_draw counter

pvrgpu_pco.c — PCO Shader Compilation
R: vertex NIR, fragment NIR, vertex format
Profile-gated compilation (fail-closed):
compile_conditionals: 1 attr R32G32B32_FLOAT, RGBA8 RT
compile_lit_mesh: 2 attrs (pos+normal), 32-DWORD CB
compile_texture: 3 attrs (pos+normal+UV), 32-DWORD CB, 1 sampler
W: pvrgpu_pco_graphics_binary (VS+FS PCO bytes + ABI)
Counter: pco_compile_* events with diagnostics

pvrgpu_cmd.c — Command Serialization + SystemC Submit
R: command struct (clear/triangle/indexed_quad/textured/pco/seq)
Output path 1: PVRGPU_DRIVER_COMMAND_OUT
-> fprintf key=value text format to driver-command.txt
Output path 2: PVRGPU_SYSTEMC_API_LIB
-> dlopen(library) -> dlsym("pvrgpu_systemc_submit_driver_command")
-> fill pvrgpu_systemc_driver_command struct
-> call submit function (synchronous execution)
W: driver-command.txt and/or SystemC model execution

pvrgpu_counter.c — Counter Event Logging
所有 module 在重要操作時呼叫 pvrgpu_counter_event(f)
Output: append "schema=pvrgpu.driver-counter.v1 producer=...
event=<name> <details>" to PVRGPU_DRIVER_COUNTER_OUT

3. pvrgpu_screen — pipe_screen 實作

pvrgpu_screen — Block Diagram

pvrgpu_create_screen(winsys, config)
Allocate pvrgpu_screen, wire vtable:
get_vendor, get_name -> "PvrGPU"
is_format_supported -> format decision table
context_create -> pvrgpu_create_context
fence_reference -> store pointer (no-op)
fence_finish -> always return true (synchronous)
Init resource functions, shader caps, screen caps

Capability Advertisement
GLSL 4.00, ESSL 3.10
max_texture_2d_size = 4096
max_render_targets = 8
Supported vertex formats: R32F, R32G32F, R32G32B32F
Supported color formats: RGBA8, BGRA8, RGB565, etc.
Supported depth formats: Z16, Z24X8, Z32F, etc.
NIR + TGSI shader IR
User vertex buffers = true

Pseudocode

```
FUNCTION pvrgpu_create_screen(winsys, config):
    screen = CALLOC(pvrgpu_screen)
    screen.winsys = winsys
    screen.vtable = {
        get_vendor: -> "PvrGPU"
        get_name: -> "PvrGPU SystemC Gallium bring-up"
        is_format_supported: pvrgpu_is_format_supported
        context_create: pvrgpu_create_context
        fence_reference: store pointer (no real fence)
        fence_finish: always true (synchronous flush)
        destroy: FREE(screen)
    }
    init_resource_functions(screen)
    init_shader_caps(screen)   // NIR+TGSI, 16384 insns, 32 I/O
    init_screen_caps(screen)   // GLSL400, ESSL310, 4096 textures
    RETURN screen

FUNCTION pvrgpu_is_format_supported(format, target, samples, bind):
    IF target == BUFFER:
        RETURN is_vertex_format(format)
    IF target NOT IN {2D, 2D_ARRAY, 3D, CUBE, CUBE_ARRAY}:
        RETURN false
    IF is_color_format(format):
        RETURN bind IN {SAMPLER_VIEW, RENDER_TARGET, DISPLAY_TARGET, BLENDABLE}
    IF is_depth_stencil_format(format):
        RETURN bind IN {DEPTH_STENCIL, SAMPLER_VIEW}
    RETURN false
```

---

4. pvrgpu_state — State Object Management

pvrgpu_state — Block Diagram

Shader State
create_vs/fs/gs/tcs/tes_state(shader_state):
Clone NIR or store TGSI tokens
Record mesa_shader_stage
Optional: dump NIR to PVRGPU_NIR_DUMP_DIR
Counter: create_shader event
bind_vs/fs_state: ctx->vs/fs = shader
delete_vs/fs_state: FREE + nir_shader_destroy

Fixed-Function State Objects
create_blend_state -> pvrgpu_blend_state
create_depth_stencil_alpha_state -> pvrgpu_dsa_state
create_rasterizer_state -> pvrgpu_rasterizer_state
create_sampler_state -> pvrgpu_sampler_state
create_vertex_elements_state -> pvrgpu_ve_state
Counter: create_*/bind_*/delete_* events

Dynamic State Binding
set_vertex_buffers: ctx->vertex_buffers[] = buffers
set_constant_buffer: ctx->constant_buffers[][] = cb
set_viewport_state: ctx->viewport = vp
set_scissor_states: ctx->scissor = sc
set_blend_color: ctx->blend_color = color
set_stencil_ref: ctx->stencil_ref = ref
set_sample_mask: ctx->sample_mask = mask
set_sampler_views: ctx->sampler_views[][] = views
bind_sampler_states: ctx->samplers[][] = samplers
Counter: set_*/bind_* events

Pseudocode

```
FUNCTION create_shader_state(pipe, state):
    shader = CALLOC(pvrgpu_shader_state)
    shader.stage = state.type  // VERTEX, FRAGMENT, etc.
    IF state.ir_type == NIR:
        shader.nir = nir_shader_clone(NULL, state.nir)
        shader.has_nir = true
        dump_nir(shader)    // if PVRGPU_NIR_DUMP_DIR set
    ELSE IF state.ir_type == TGSI:
        shader.tgsi = tgsi_dup_tokens(state.tokens)
        shader.has_tgsi = true
    counter_event("create_shader", "stage=vertex ...")
    RETURN shader

FUNCTION bind_vs_state(pipe, state):
    ctx = pvrgpu_context(pipe)
    ctx->vs = (pvrgpu_shader_state *)state
    counter_event("bind_shader", "stage=vertex")

FUNCTION create_blend_state(pipe, state):
    blend = CALLOC(pvrgpu_blend_state)
    blend->state = *state    // deep copy pipe_blend_state
    counter_event("create_blend_state", ...)
    RETURN blend

FUNCTION set_vertex_buffers(pipe, start, count, buffers):
    ctx = pvrgpu_context(pipe)
    util_set_vertex_buffers(ctx->vertex_buffers, ctx->num_vertex_buffers,
                            start, count, buffers)
    counter_event("set_vertex_buffers", "count=N")
```

---

5. pvrgpu_resource — Resource Management

pvrgpu_resource — Block Diagram

resource_create(screen, template):
Allocate pvrgpu_resource + CPU-backed data[]
Compute stride, layer_stride, level offsets/strides
IF display_target: allocate via sw_winsys
Counter: resource_create event

buffer_map / texture_map:
Return direct pointer to data + offset
IF display_target: map via sw_winsys
Counter: resource_map event

transfer_unmap:
IF display_target: unmap sw_winsys
Counter: resource_unmap event

buffer_subdata / texture_subdata:
memcpy(resource.data + offset, user_data, size)
Counter: texture_subdata / buffer_subdata event

resource_copy_region / blit:
CPU-side memcpy between pvrgpu_resource instances
Format conversion for same-size color formats
Counter: resource_copy_region / blit event

resource_destroy:
FREE(data); IF display_target: destroy via winsys
Counter: resource_destroy event

Pseudocode

```
FUNCTION resource_create(screen, template):
    res = CALLOC(pvrgpu_resource)
    res.base = *template
    block_size = format_block_size(template.format)
    res.stride = align(template.width * block_size, alignment)
    res.size = stride * height * depth * array_size
    IF template.bind & DISPLAY_TARGET:
        res.displaytarget = winsys->displaytarget_create(...)
    ELSE:
        res.data = malloc(res.size)
        memset(res.data, 0, res.size)
    counter_event("resource_create", "width=W height=H format=F size=S")
    RETURN res

FUNCTION buffer_map(pipe, resource, level, usage, box, transfer):
    res = pvrgpu_resource(resource)
    offset = box.x * format_block_size
    *transfer = create_transfer(pipe, resource, level, usage, box)
    RETURN res.data + offset

FUNCTION texture_subdata(pipe, resource, level, usage, box, data, stride, layer_stride):
    res = pvrgpu_resource(resource)
    FOR EACH layer IN box.depth:
        dst_offset = level_offset + layer * layer_stride + box.y * res.stride + box.x * block_size
        memcpy(res.data + dst_offset, data + src_offset, row_bytes * box.height)
    counter_event("texture_subdata", "level=L box=WxH")
```

---

6. pvrgpu_clear — Clear Path

pvrgpu_clear — Block Diagram

pvrgpu_clear(pipe, buffers, color, depth, stencil)
Check: is this a full-frame color-only clear?
- No scissor (or scissor covers entire FB)
- PIPE_CLEAR_COLOR0 set
- Format is supported (RGBA8, BGRA8, RGB565, etc.)
- Framebuffer matches requested RDC output

IF full-frame color clear:
Pack color components to uint32 bits
Build pvrgpu_clear_color_command:
case_name, frame, width, height, format
clear_color_bits[4]
Write to PVRGPU_DRIVER_COMMAND_OUT (text file)
AND/OR submit via SystemC API
CPU clear: memset framebuffer resource data
ELSE:
CPU-only clear (memset resource data)
Counter: unsupported_clear event

Depth/Stencil Clear Tracking
pvrgpu_note_full_depth_clear_one():
Record that a depth=1.0 full-frame clear happened
(used by draw path to validate depth state)
pvrgpu_invalidate_full_depth_clear():
Any non-clear depth write invalidates the record

Pseudocode

```
FUNCTION pvrgpu_clear(pipe, buffers, color, depth, stencil):
    ctx = pvrgpu_context(pipe)
    fb = ctx->framebuffer

    // CPU-side clear (always executed)
    IF buffers & PIPE_CLEAR_COLOR0:
        pack color to bytes based on format
        FOR EACH pixel in fb.cbufs[0]:
            memset(pixel, packed_color, block_size)

    IF buffers & PIPE_CLEAR_DEPTHSTENCIL:
        FOR EACH pixel in fb.zsbuf:
            write depth/stencil value

    // Driver command emit (only for supported full-frame color clear)
    IF is_full_frame_color_clear(fb, buffers, scissor):
        cmd = {
            case_name: env(PVRGPU_RDC_CASE_NAME) or "phase1.clear.gallium"
            frame: 0
            width: fb.width, height: fb.height
            format: pipe_format_to_string(fb.cbufs[0]->format)
            clear_color_bits: float_to_bits(r, g, b, a)
        }
        path = env(PVRGPU_DRIVER_COMMAND_OUT)
        IF path:
            pvrgpu_write_clear_color_command(path, &cmd)
        submit_systemc_api(&cmd)
        counter_event("clear_color", "width=W height=H r=R g=G b=B a=A")
    ELSE:
        counter_event("unsupported_clear", ...)
```

7. pvrgpu_context — Draw Path (核心分派)

pvrgpu_draw_vbo — Draw Dispatch Decision Tree

Step 1: Case Suppression Check
IF case_suppresses_draw_commands():
counter("draw_suppressed"); RETURN

Step 2: PCO Lit-Mesh Profile Match
IF draw_matches_lit_mesh(ctx, info, draws):
Validate: 2 attrs (pos R32G32B32F + normal)
VS + FS NIR hash match known profile
Compile NIR -> PCO via pvrgpu_pco_compile_lit_mesh
Emit pvrgpu_draw_pco_triangles_command
Include: raw VBO, VS/FS PCO bytes, shared regs
Counter: draw_pco_lit_mesh
RETURN

Step 3: PCO Texture Profile Match
IF draw_matches_texture_pco(ctx, info, draws):
Validate: 3 attrs (pos+normal+UV), sampler2D
Compile NIR -> PCO via pvrgpu_pco_compile_texture
Emit draw_pco_triangles_command with texture data
Counter: draw_pco_texture
RETURN

Step 4: PCO Conditionals Profile Match
IF draw_matches_conditionals(ctx, info, draws):
Validate: 1 attr R32G32B32F, specific NIR shape
Compile NIR -> PCO via pvrgpu_pco_compile_conditionals
Emit draw_pco_triangles_command
Counter: draw_pco_conditionals
RETURN

Step 5: Textured Triangles (glmark2 effect2d)
IF draw_is_observable_textured_triangle(ctx, info):
Validate: 6 vertices, texture sampler bound
Emit pvrgpu_draw_textured_triangles_command
Include: vertex bits, texcoord bits, texture RGBA8
Counter: draw_textured_triangles
RETURN

Step 6: dEQP Primitive Sequence Match
IF case_prefers_draw_counter_sequence():
Match pending_primitive_sequence_profile
Emit draw_primitive_sequence_command with counters
Counter: draw_counter_sequence
RETURN

Step 7: Simple Array Triangle (GL_TRIANGLES, 3v)
IF draw_is_observable_array_triangle(ctx, info):
Validate: non-indexed, 3 vertices, GL_TRIANGLES
Emit pvrgpu_draw_triangle_command
Counter: draw_triangles
RETURN

Step 8: Indexed Quad (GL_TRIANGLES, indexed)
IF draw_is_observable_indexed_quad(ctx, info):
Validate: indexed, GL_TRIANGLES, has user indices
Emit pvrgpu_draw_indexed_quad_command
Counter: draw_indexed_quad
RETURN

Step 9: Indexed Triangle (counter only)

IF draw_is_observable_indexed_triangle(ctx, info):
Counter: draw_indexed_triangles (not yet lowered)
RETURN

Step 10: Unsupported Draw

Counter: unsupported_draw + detailed state dump
RETURN

Pseudocode

```
FUNCTION pvrgpu_draw_vbo(pipe, info, drawid_offset, indirect, draws, num_draws):
    ctx = pvrgpu_context(pipe)

    // 1. Suppression
    IF case_suppresses_draw_commands():
        counter("draw_suppressed"); RETURN

    // 2. Lit-mesh PCO profile
    IF draw_matches_lit_mesh(ctx, info, draws, &observation):
        binary = pco_compile_lit_mesh(ctx->pco_compiler, ctx->vs->nir, ctx->fs->nir, profile)
        cmd = build_pco_triangles_command(ctx, observation, binary)
        pvrgpu_write_draw_pco_triangles_command(path, &cmd)
        submit_systemc_api(&cmd)
        counter("draw_pco_lit_mesh", "profile=N count=C")
        RETURN

    // 3. Texture PCO profile
    IF draw_matches_texture_pco(ctx, info, draws, &observation):
        binary = pco_compile_texture(ctx->pco_compiler, ctx->vs->nir, ctx->fs->nir)
        cmd = build_pco_triangles_command(ctx, observation, binary)
        // Include sampled texture bytes
        cmd.sampled_texture_bytes = texture_resource.data
        pvrgpu_write_draw_pco_triangles_command(path, &cmd)
        counter("draw_pco_texture", ...)
        RETURN

    // 4. Conditionals PCO profile
    IF draw_matches_conditionals(ctx, info, draws, &observation):
        binary = pco_compile_conditionals(ctx->pco_compiler, vs_nir, fs_nir, vertex_format)
        cmd = build_pco_triangles_command(ctx, observation, binary)
        submit_systemc_api(&cmd)
        counter("draw_pco_conditionals", ...)
        RETURN

    // 5. Textured triangles (glmark2)
    IF draw_is_observable_textured_triangle(ctx, info, draws):
        cmd = build_textured_triangles_command(ctx, draws, observation)
        pvrgpu_write_draw_textured_triangles_command(path, &cmd)
        counter("draw_textured_triangles", ...)
        RETURN

    // 6. dEQP primitive sequence
    IF case_prefers_draw_counter_sequence():
        cmd = build_primitive_sequence_command(ctx, profile)
        pvrgpu_write_draw_primitive_sequence_command(path, &cmd)
        counter("draw_counter_sequence", ...)
        RETURN

    // 7. Simple array triangle
    IF draw_is_observable_array_triangle(ctx, info, draws):
        cmd = build_triangle_command(ctx, draws)
        pvrgpu_write_draw_triangle_command(path, &cmd)
        counter("draw_triangles", ...)
        RETURN

    // 8. Indexed quad
    IF draw_is_observable_indexed_quad(ctx, info, draws, &observation):
        cmd = build_indexed_quad_command(ctx, observation)
        pvrgpu_write_draw_indexed_quad_command(path, &cmd)
```

```
    counter("draw_indexed_quad", ...)
    RETURN

// 9. Indexed triangle (counter only)
IF draw_is_observable_indexed_triangle(ctx, info, draws):
    counter("draw_indexed_triangles", ...)
    RETURN

// 10. Unsupported
counter("unsupported_draw", "reason=unsupported_state")
```

8. pvrgpu_pco — PCO Shader Compiler Bridge

pvrgpu_pco — Block Diagram

pvrgpu_pco_compiler_create():
Initialize PCO context for target "gx6250"
Set up pvr_device_info + runtime_info
Create pco_ctx via public Mesa PCO API

Profile-Gated Compilation (fail-closed)
Each profile validates exact NIR shape before
invoking the PCO compiler pipeline.
compile_conditionals:
Input: 1 attr (R32G32B32_FLOAT), RGBA8 RT
VS: uniform-driven transformations
FS: uniform-driven conditionals (fract/compare)
compile_lit_mesh:
Input: 2 attrs (position + normal), 32-DWORD CB
VS: MVP transform + lighting normal transform
FS: scalar or vec3 varying consumption
Profiles: build/bump/shading
compile_texture:
Input: 3 attrs (position + normal + UV)
VS: MVP transform + varying passthrough
FS: sample texture * intensity varying

Output: pvrgpu_pco_graphics_binary
vertex.data[] : VS PCO machine code bytes
vertex.abi : temps, inputs, outputs, shareds, etc.
fragment.data[] : FS PCO machine code bytes
fragment.abi : temps, coefficients, shareds, etc.
position_output_start/count : VS->FS clip pos linkage
varying_output_start/count : VS->FS varying linkage
fragment_texture_descriptor_start/count/stride

9. pvrgpu_cmd — Command Serialization + SystemC Submit

pvrgpu_cmd — Block Diagram

Text File Output
Path: PVRGPU_DRIVER_COMMAND_OUT
Format: "pvrgpu.driver-command.v1" key=value text
write_clear_color_command:
schema command=clear_color case=... frame=0
width=W height=H format=F
clear_r=0xRR clear_g=0xGG clear_b=0xBB clear_a=0xAA
write_draw_triangle_command:
command=draw_triangle vertex0..2 fragment_color
write_draw_pco_triangles_command:
command=draw_pco_triangles vertex_data_hex
vertex_pco_hex fragment_pco_hex
vertex_shared[] fragment_shared[]
ABI fields: temps, inputs, outputs, coefficients

SystemC API Submit
Path: PVRGPU_SYSTEMC_API_LIB (dlopen)
1. dlopen(library_path, RTLD_NOW | RTLD_GLOBAL)
2. dlsym("pvrgpu_systemc_submit_driver_command")
3. Fill pvrgpu_systemc_driver_command struct
(version, schema, producer, command, all fields)
4. Fill pvrgpu_systemc_submit_info
(jsonl_path, stderr_path, outdir, memory_mode)
5. Call submit_fn(info, error, error_size)
6. Synchronous: model runs and returns
Counter: systemc_api_submit / systemc_api_error

Pseudocode

```
FUNCTION pvrgpu_write_clear_color_command(path, cmd):
    file = fopen(path, "w")
    fprintf(file, "schema=%s\n", PVRGPU_DRIVER_COMMAND_SCHEMA)
    fprintf(file, "producer=%s\n", PVRGPU_DRIVER_COMMAND_PRODUCER)
    fprintf(file, "command=clear_color\n")
    fprintf(file, "case=%s\n", cmd->case_name)
    fprintf(file, "width=%u height=%u\n", cmd->width, cmd->height)
    fprintf(file, "format=%s\n", cmd->format)
    FOR i IN 0..3:
        fprintf(file, "clear_%c=0x%08x\n", "rgba"[i], cmd->clear_color_bits[i])
    fclose(file)

FUNCTION pvrgpu_submit_systemc_api(command):
    library_path = env(PVRGPU_SYSTEMC_API_LIB)
    IF !library_path: counter("systemc_api_disabled"); RETURN true

    handle = dlopen(library_path, RTLD_NOW | RTLD_GLOBAL)
    submit_fn = dlsym(handle, "pvrgpu_systemc_submit_driver_command")

    info = {
        version: PVRGPU_SYSTEMC_API_VERSION,
        command: command,
        jsonl_path: env(PVRGPU_SYSTEMC_JSONL_OUT),
        outdir: env(PVRGPU_SYSTEMC_OUTDIR),
        memory_mode: env(PVRGPU_MODEL_MEMORY_MODE)
    }
    result = submit_fn(&info, error, sizeof(error))
    counter("systemc_api_submit", "result=R")
    RETURN result == 0
```

---

10. pvrgpu_counter — Event Logging

pvrgpu_counter — Block Diagram

```
pvrgpu_counter_event(event, details):
path = env(PVRGPU_DRIVER_COUNTER_OUT)
IF !path: RETURN (counter disabled)
file = fopen(path, "a") // append mode
fprintf(file,
"schema=pvrgpu.driver-counter.v1 "
"producer=pvrgpu-gallium-driver "
"event=%s %s\n", event, details)
fclose(file)
```

11. Environment Variable Control

變數	用途
`GALLIUM_DRIVER=pvrgpu`	Mesa software-loader 選擇此 driver
`PVRGPU_DRIVER_COMMAND_OUT`	driver command text file 輸出路徑
`PVRGPU_DRIVER_COUNTER_OUT`	driver counter JSONL 輸出路徑
`PVRGPU_SYSTEMC_API_LIB`	SystemC model shared library 路徑 (dlopen)
`PVRGPU_SYSTEMC_JSONL_OUT`	SystemC model JSONL 輸出路徑
`PVRGPU_SYSTEMC_OUTDIR`	SystemC model 輸出目錄
`PVRGPU_SYSTEMC_STDERR_OUT`	SystemC model stderr 輸出路徑
`PVRGPU_MODEL_MEMORY_MODE`	SystemC model 記憶體模式
`PVRGPU_RDC_CASE_NAME`	當前 test case 名稱
`PVRGPU_RDC_TRACE_DRAW_ACTIONS`	追蹤 draw action 計數
`PVRGPU_NIR_DUMP_DIR`	NIR shader dump 目錄
`PVRGPU_DEBUG_FORMAT_SUPPORT`	開啟 format 支援 debug 記錄

12. Supported Driver Command Types

Command	觸發條件	包含資料
`clear_color`	全屏 color clear	width, height, format, RGBA bits
`draw_triangle`	3 vertex non-indexed GL_TRIANGLES	vertex XY bits, fragment RGBA bits
`draw_indexed_quad`	indexed GL_TRIANGLES (quad)	index_count, unique_vertices, primitive_count, texel_fetches
`draw_textured_triangles`	6v textured (glmark2 effect2d)	vertex/texcoord bits, texture RGBA8 path
`draw_pco_triangles`	PCO profile match	raw VBO, VS/FS PCO binary, shared regs, ABI, rasterizer state
`draw_primitive_sequence`	dEQP counter sequence	all pipeline stage counters, framebuffer RGBA8 path

13. Fail-Closed Design Pattern

Fail-Closed Design

Driver 採用 fail-closed 設計：
1. 每個 draw profile 有精確的 state 驗證
不匹配 -> 直接 unsupported_draw counter
2. PCO 編譯只接受已知的 NIR 形狀
未知 shader -> 編譯拒絕，不發 command
3. CPU-side 總是執行 (clear memset, readback)
確保 glReadPixels 結果正確
4. Driver command 只在完全匹配時才 emit
不匹配的 draw 只記錄 counter，不送 model
5. 一次只 emit 一個 driver draw command
後續相同 case 的 draw 只記 counter